

MUSIC PRO COURIER

Evaluación 3: Aplicación web con CRUD

NOMBRE: Pablo Martínez
CARRERA: Ingeniería en Ciberseguridad
ASIGNATURA: Programación Back End (TI3041)
PROFESOR: Benjamin Mora
FECHA: 8 de noviembre de 2026
REPOSITORIO: https://github.com/maxsteel19/musicpro-courier



1 Introducción

Music Pro Tour traslada instrumentos y equipos de audio entre sus sucursales. En la Evaluación 2 se modeló este proceso (sucursales, paquetes, envíos, incidencias y notificaciones) y se administró con Django Admin. En esta **Evaluación 3** el caso continúa: se desarrolló un **sistema web administrativo propio, sin Django Admin**, conectado a **MySQL**, con autenticación de usuarios, control de acceso mediante decoradores, mantención de usuarios y de todas las entidades del caso, y una página de **reportes construidos exclusivamente con el ORM de Django**.

Para el informe de consultas ORM se trabajó con el proyecto **Transporte** (vehículos, conductores y viajes), que se integró de forma natural al courier: cada envío viaja en un viaje que tiene asignados un vehículo y un conductor.

El código fuente completo está en GitHub: **https://github.com/maxsteel19/musicpro-courier** (rama **main**).

1.1 Resumen de lo implementado

- Conexión local con **MySQL** (credenciales en un archivo `.env`, fuera del repositorio).
- Inicio y cierre de sesión con `django.contrib.auth` y plantillas propias.
- Todas las vistas privadas protegidas con `@login_required`; gestión de usuarios protegida además con el decorador propio `@administrador_requerido`.
- Mantención de usuarios: crear, consultar, modificar, cambiar contraseña, **desactivar/activar** y eliminar.
- CRUD de **9 entidades**: sucursales, paquetes, envíos, incidencias, notificaciones, tipos de vehículo, vehículos, conductores y viajes.
- **20 reportes** ORM del proyecto Transporte, con resultado, código ORM y SQL generado como evidencia.
- Panel de control con indicadores y gráficos filtrables por período, y **60 pruebas automáticas** que pasan en SQLite y MySQL.

2 Repositorio y ejecución

2.1 Tecnologías

Componente	Detalle
Lenguaje y framework	Python 3.12+ · Django 6.1 (compatible con Django 5.2 LTS)
Base de datos	MySQL 8.4+ o MariaDB 10.11+ (driver PyMySQL)
Interfaz	Plantillas de Django, HTML, CSS, JavaScript y Bootstrap 5 (incluido en el proyecto, funciona sin internet)
Datos de prueba	Faker (configuración chilena: RUT, patentes, ciudades)
Pruebas	django.test (60 pruebas automáticas)



2.2 Instalación (Windows)

```
git clone https://github.com/maxsteel19/musicpro-courier.git
cd musicpro-courier
python -m venv env
env\Scripts\activate
pip install -r requirements.txt
```

Crear la base de datos en MySQL (Workbench o consola `mysql -u root -p`):

```
CREATE DATABASE musicpro_courier CHARACTER SET utf8mb4 COLLATE utf8mb4_unicode_ci;
```

Copiar `.env.example` como `.env`, completar usuario y clave de MySQL, y luego:

```
python manage.py migrate
python manage.py seed_courier        # carga datos de prueba realistas
python manage.py createsuperuser     # primer administrador
python manage.py runserver
```

Página	URL
Sitio público	http://127.0.0.1:8000/
Inicio de sesión	http://127.0.0.1:8000/cuentas/login/
Panel de control (requiere sesión)	http://127.0.0.1:8000/panel/
Reportes ORM (requiere sesión)	http://127.0.0.1:8000/reportes/

2.3 Estructura principal

Archivo / carpeta	Contenido
`musicpro/settings.py`	Configuración: MySQL, sesión, seguridad, `LOGIN_URL`
`courier/models.py`	Modelos del courier y del transporte
`courier/forms.py`	Formularios con validaciones (RUT, patente, fechas)
`courier/decorators.py`	Decorador `@administrador_requerido`
`courier/views/`	Vistas: cuentas, panel, gestión (CRUD), usuarios y reportes
`courier/reportes.py`	Las 20 consultas del informe ORM
`courier/tests/`	Pruebas automáticas
`templates/`	Plantillas HTML del administrador propio
`docs/GUIA_DEFENSA.md`	Explicación del código para la defensa

3 Parte 1: Sistema web administrativo

3.1 Requisitos y cómo se cumplen

Requisito	Cómo se cumple	Dónde
Conexión con MySQL	Motor `django.db.backends.mysql`; credenciales leídas desde `.env`	`settings.py`



Requisito	Cómo se cumple	Dónde
Autenticación	`LoginView` y `LogoutView` de Django con plantillas propias	`views/cuentas.py`
Decoradores	`@login_required` en todas las vistas privadas; `@administrador_requerido` en usuarios	`views/*.py`
Protección de funcionalidades	Sin sesión, toda URL privada (incluso POST y JSON) redirige al login	`tests/test_autenticacion.py`
Mantención de usuarios	Crear, listar, editar, cambiar clave, desactivar/activar y eliminar	`views/usuarios.py`
Entidades del caso	CRUD de 9 entidades con búsqueda, filtros y paginación	`views/gestion.py`
Uso del ORM	Todas las consultas con el ORM; sin SQL escrito a mano	`views/`, `reportes.py`
Interfaz web funcional	Panel con menú lateral, formularios validados y mensajes	`templates/`
Persistencia	Cada operación se refleja en MySQL (verificado con pruebas)	`tests/test_crud.py`
Sin Django Admin	`django.contrib.admin` retirado de `INSTALLED_APPS` y de las URLs	`settings.py`, `urls.py`

3.2 Conexión con MySQL

La conexión se configura en `musicpro/settings.py`. Los datos sensibles se leen desde variables de entorno (archivo `.env`, incluido en `.gitignore`), por lo que no quedan publicados en GitHub:

```
DATABASES = {
    'default': {
        'ENGINE': 'django.db.backends.mysql',
        'NAME': os.environ.get('DB_NAME', 'musicpro_courier'),
        'USER': os.environ.get('DB_USER', 'root'),
        'PASSWORD': os.environ.get('DB_PASSWORD', ''),
        'HOST': os.environ.get('DB_HOST', '127.0.0.1'),
        'PORT': os.environ.get('DB_PORT', '3306'),
        'OPTIONS': {'charset': 'utf8mb4'},
    }
}
```

3.3 Autenticación y sesión

El inicio de sesión usa `LoginView` de Django: valida la contraseña contra el hash guardado, crea una sesión nueva (cambiando el identificador para evitar la fijación de sesión) y guarda en ella el usuario autenticado. Si alguien intenta abrir una página privada sin sesión, llega al login con el aviso "Debe iniciar sesión" y, tras ingresar, vuelve a la página que pidió (parámetro `next`, que solo acepta URLs del propio sitio).

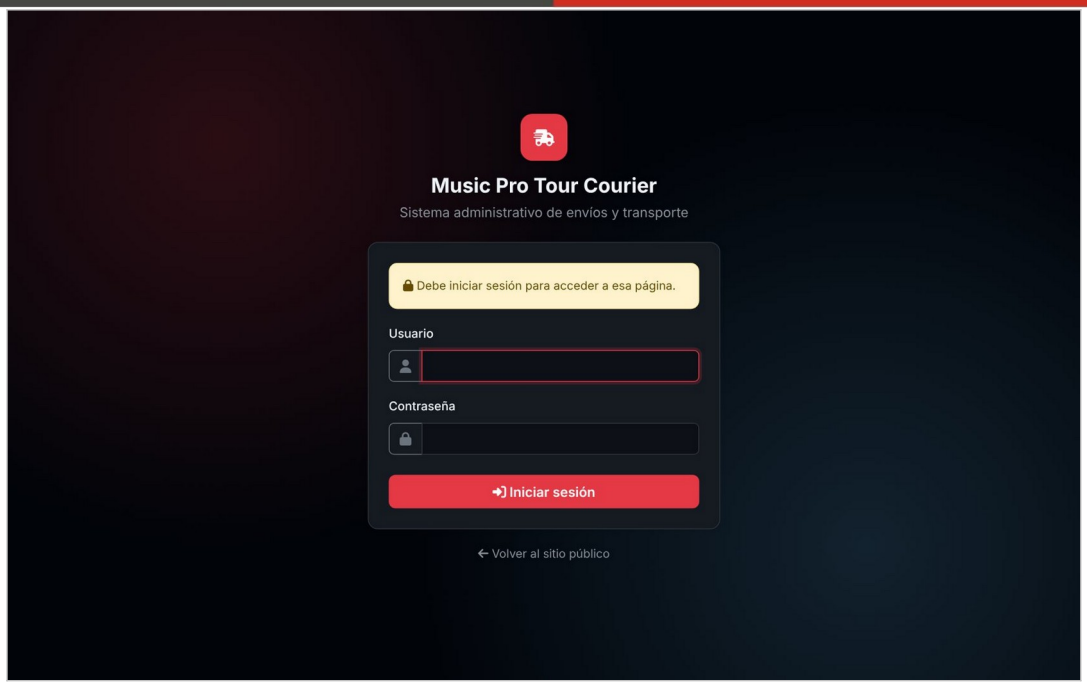


Figura 1. Al abrir /vehiculos/ sin sesión, el sistema redirige al login.

El cierre de sesión se hace con un formulario **POST** con token CSRF (Django ya no acepta cerrar sesión por GET). Al cerrar sesión la sesión se elimina y las páginas privadas no se guardan en la caché del navegador (cabecera `Cache-Control: no-store`), por lo que el botón "Atrás" no vuelve a mostrar datos.

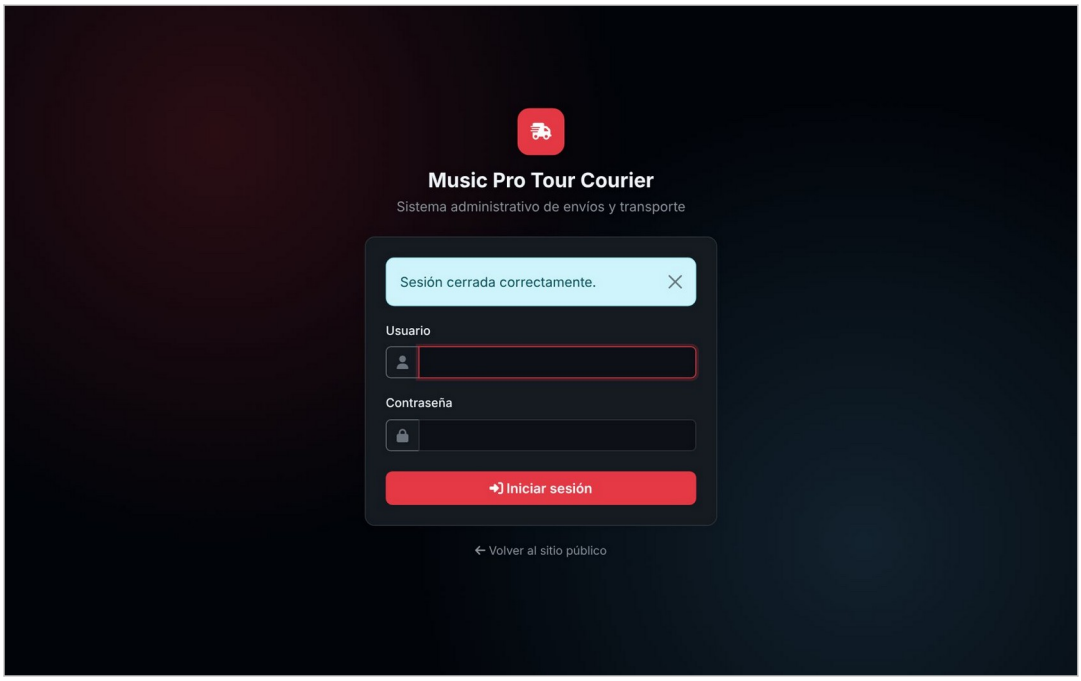


Figura 2. Mensaje tras cerrar sesión. Al volver a una URL privada se exige iniciar sesión otra vez.



3.4 Control de acceso con decoradores

Cada vista privada está decorada con `@login_required`. Si el usuario no inició sesión, Django responde con una redirección al login y **la vista no se ejecuta**, de modo que tampoco se puede modificar la base de datos con un POST directo:

```
@login_required
def vehiculo_lista(request):
    vehiculos = (Vehiculo.objects.select_related('tipo_vehiculo')
                 .annotate(total_viajes=Count('viajes')).order_by('patente'))
    ...
```

La mantención de usuarios exige además el rol de administrador. Para eso se creó un decorador propio que combina `login_required` con una revisión del rol y responde **403 (acceso denegado)** a los operadores:

```
def administrador_requerido(vista):
    @wraps(vista)
    @login_required
    def _vista_protegida(request, *args, **kwargs):
        if not (request.user.is_staff or request.user.is_superuser):
            raise PermissionDenied
        return vista(request, *args, **kwargs)
    return _vista_protegida
```

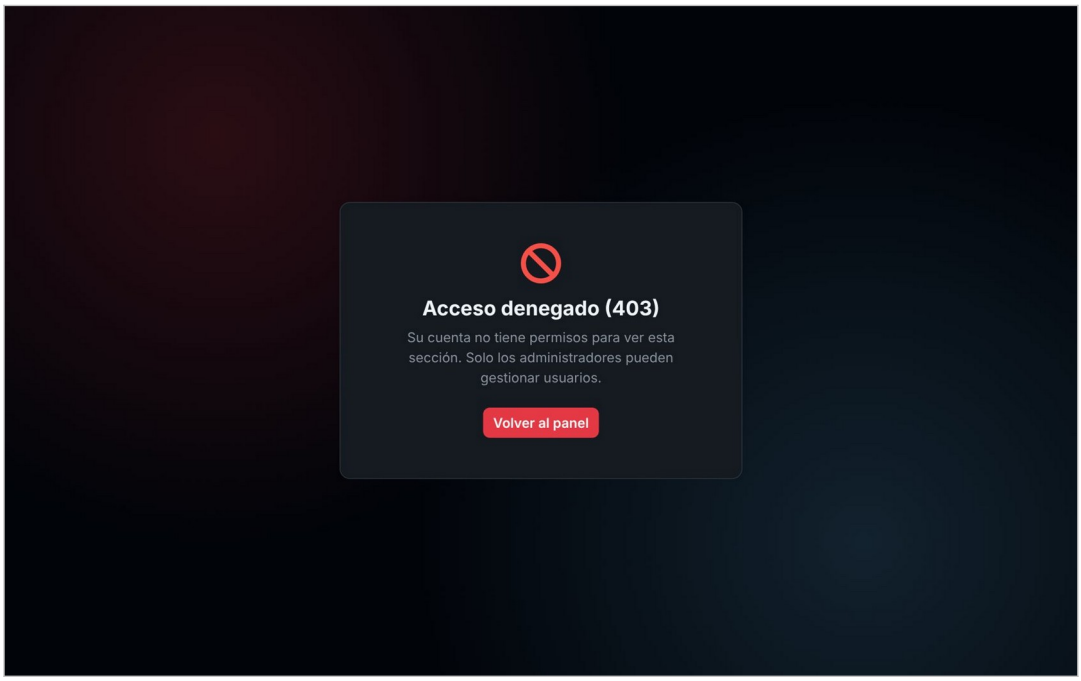


Figura 3. Un operador que intenta entrar a /usuarios/ recibe "Acceso denegado (403)".

3.5 Mantención de usuarios

Se implementó un mantenedor propio (sin Django Admin), disponible solo para administradores:

- **Crear** usuarios con `UserCreationForm`: la contraseña se guarda cifrada (PBKDF2) y se validan largo mínimo, contraseñas comunes y solo números.
- **Consultar** con búsqueda por usuario, nombre o correo, y filtros por rol y estado.



- **Modificar** datos y rol (Administrador u Operador) y **cambiar la contraseña** de otro usuario.
- **Desactivar / activar** una cuenta (bloquea el acceso sin perder su historial) o **eliminarla**.
- Protecciones: nadie puede desactivarse, eliminarse ni quitarse su propio rol, y solo un superusuario puede modificar a otro superusuario.

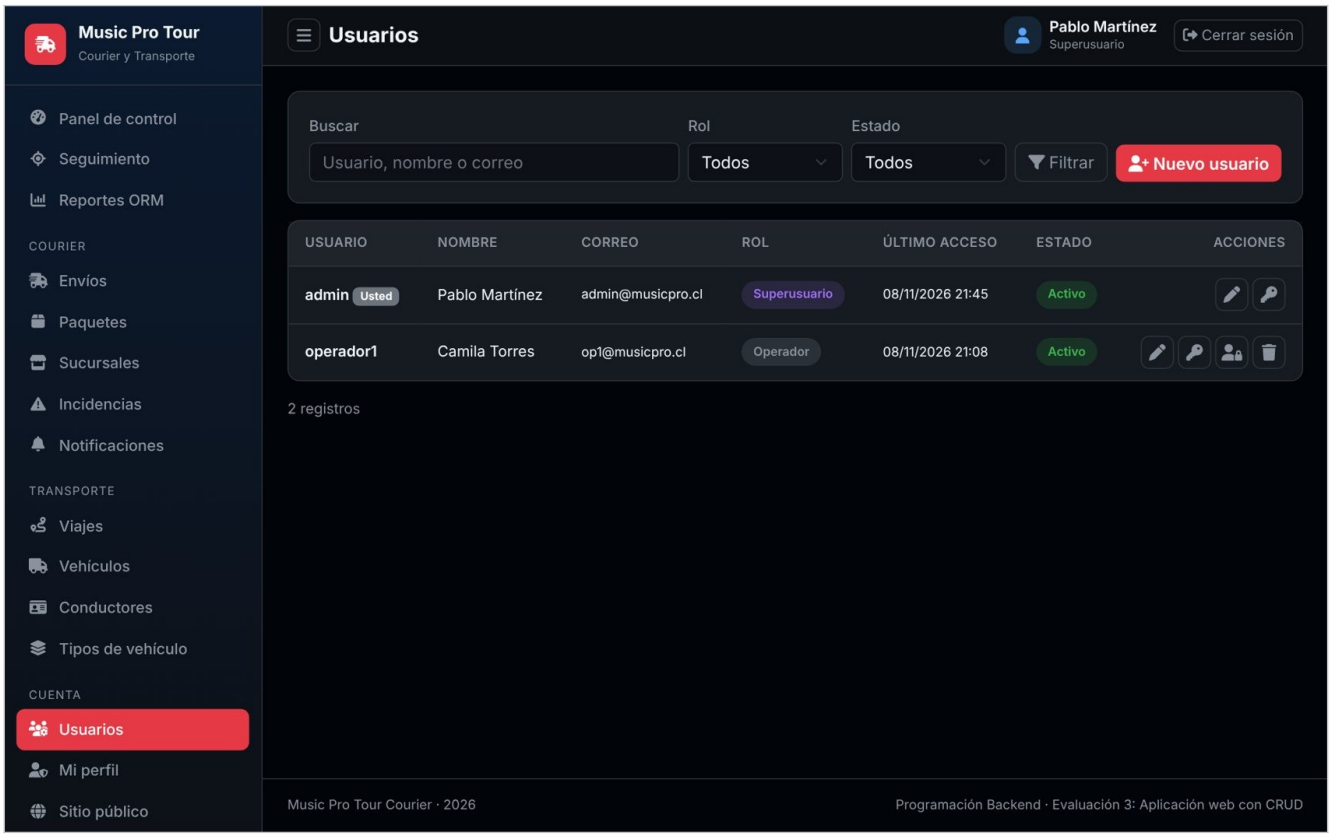


Figura 4. Listado de usuarios con acciones de editar, cambiar clave, desactivar y eliminar.

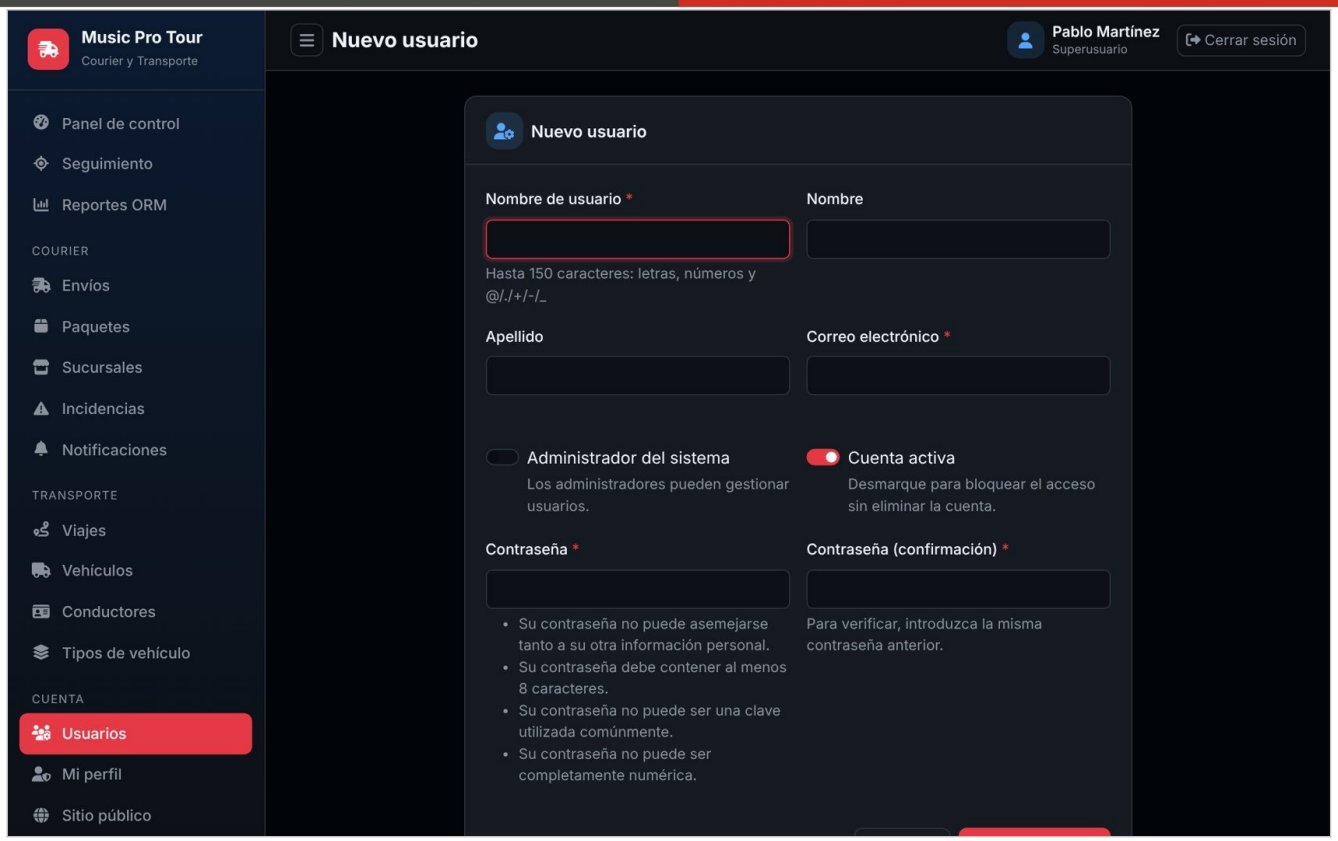


Figura 5. Formulario de creación de usuario con las reglas de contraseña de Django.

3.6 Mantención de las entidades del caso

Módulo	Entidad	Operaciones y validaciones
Courier	Sucursales	CRUD, búsqueda, filtro activas/inactivas, conteo de envíos
Courier	Paquetes	CRUD, filtro por estado; código normalizado
Courier	Envíos	CRUD y detalle; origen distinto del destino; guarda el usuario que lo creó
Courier	Incidencias	CRUD; guarda el usuario que la reportó
Courier	Notificaciones	CRUD, filtro por canal
Transporte	Tipos de vehículo	CRUD, conteo de vehículos
Transporte	Vehículos	CRUD; patente chilena validada y normalizada (BCDF-12 / AB-1234)
Transporte	Conductores	CRUD; RUT validado con dígito verificador (módulo 11)
Transporte	Viajes	CRUD y detalle; llegada posterior a la salida; conductor activo



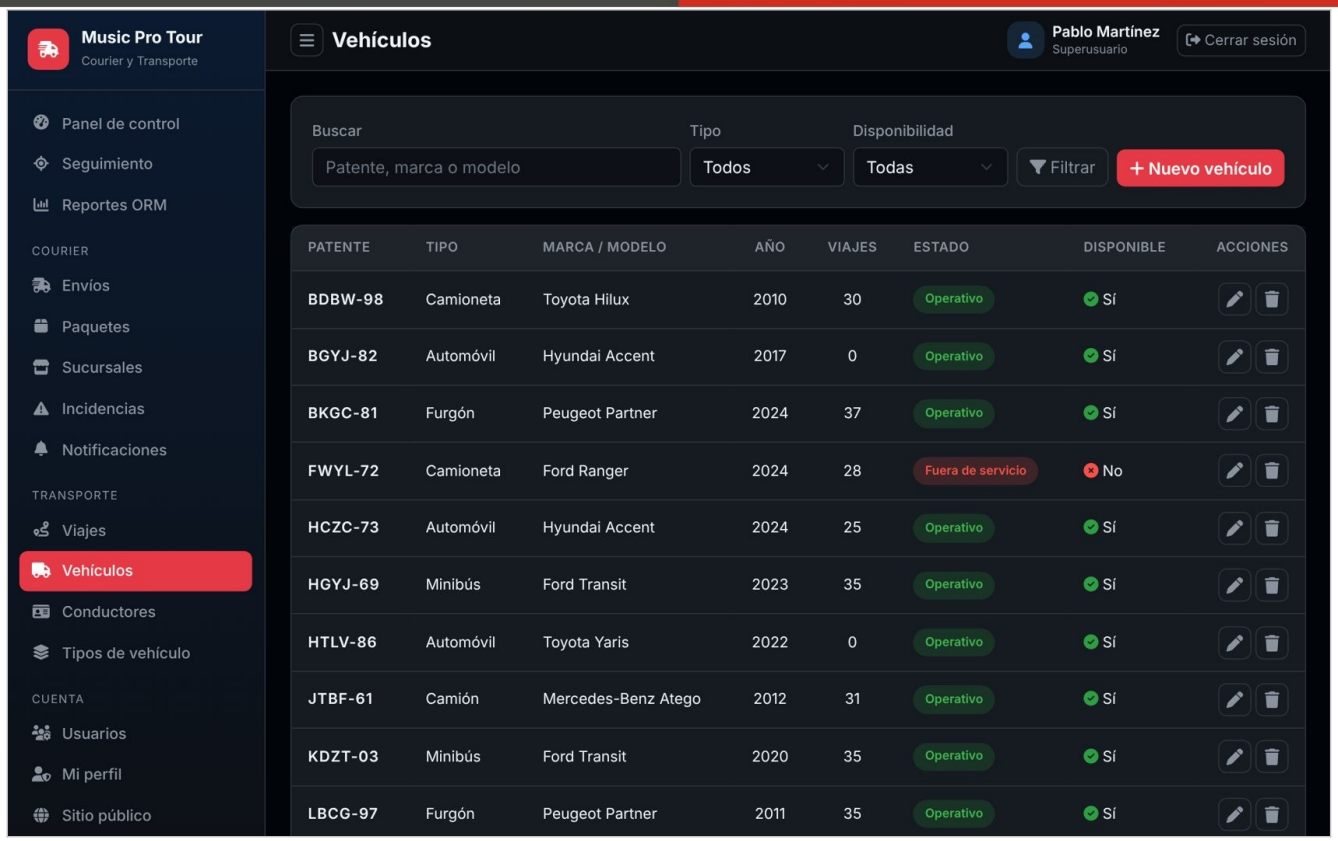


Figura 6. Listado de vehículos con búsqueda, filtros, paginación y acciones.

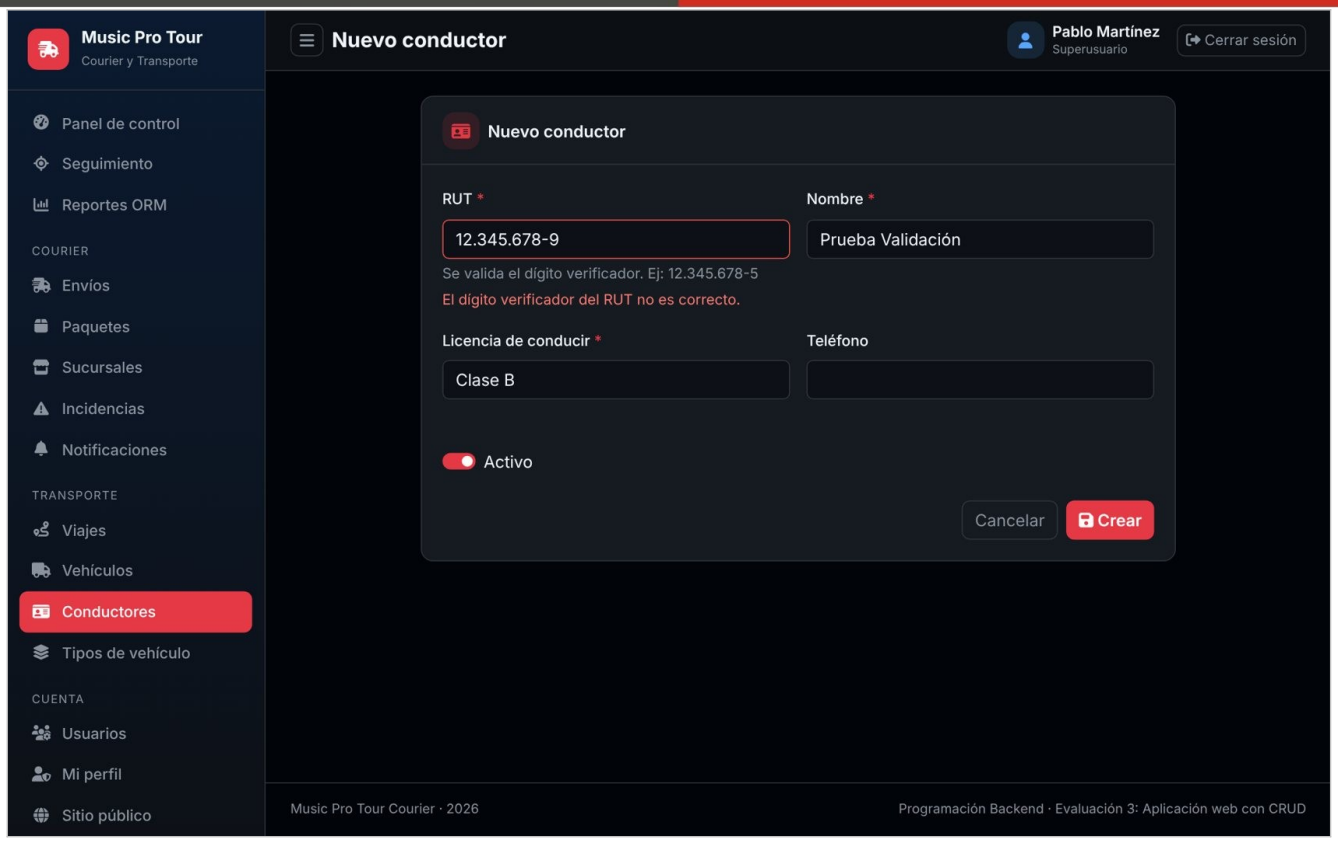


Figura 7. Validación del formulario: RUT con dígito verificador incorrecto.

Para mantener la integridad, las relaciones usan `on_delete=PROTECT`: un registro con datos asociados no se puede eliminar y el sistema informa el motivo sin modificar la base de datos.



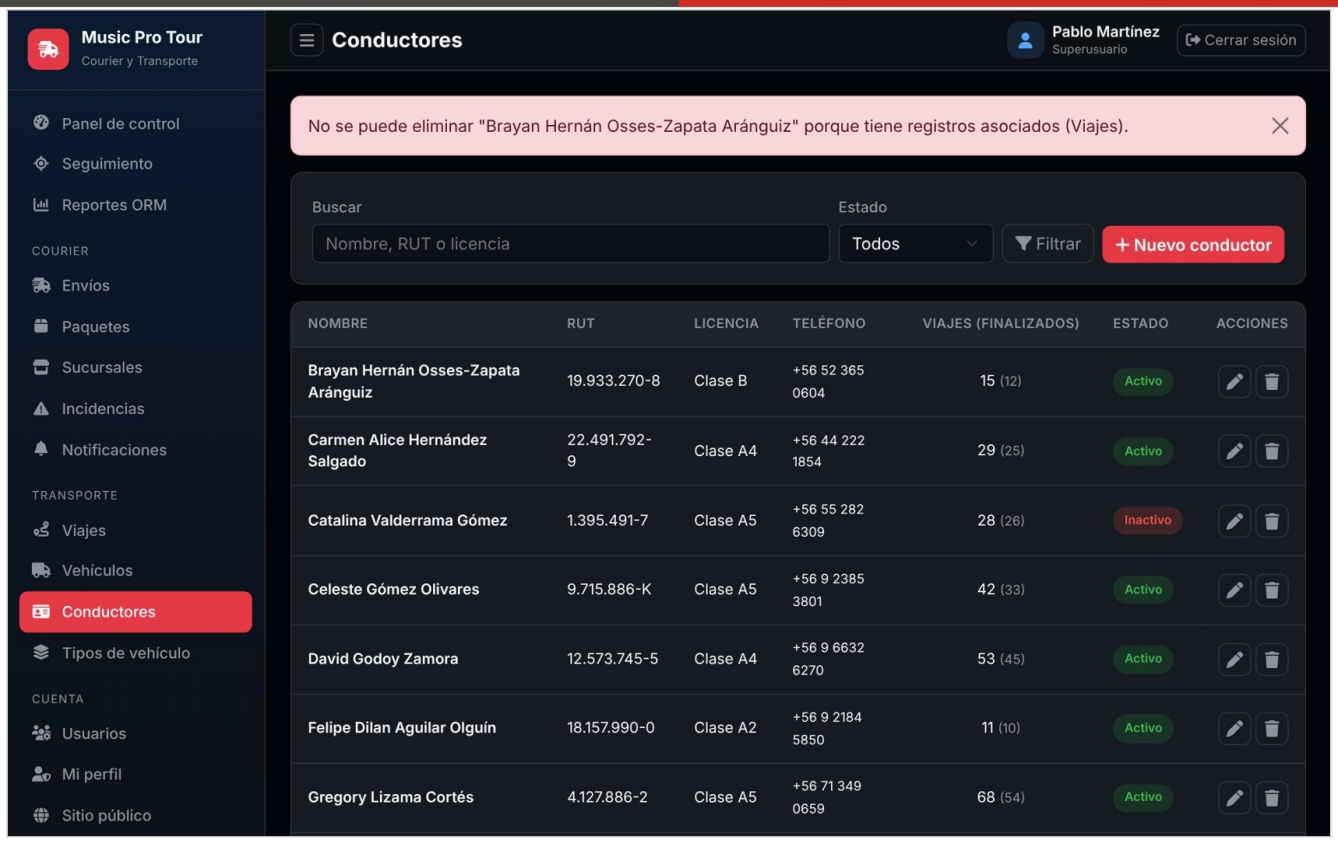


Figura 8. Intento de eliminar un conductor con viajes registrados: la operación se rechaza.

3.7 Uso del ORM de Django

Necesidad	Ejemplo con el ORM
Búsqueda en varios campos	`Conductor.objects.filter(Q(nombre__icontains=q) | Q(rut__icontains=q))`
Evitar consultas repetidas (N+1)	`Viaje.objects.select_related('vehiculo', 'conductor')`
Contar relacionados	`Vehiculo.objects.annotate(total_viajes=Count('viajes'))`
Agregación condicional	`Count('id', filter=Q(estado='finalizado'))`
Operar entre columnas	`Avg(F('fecha_llegada') - F('fecha_salida'))`
Orden lógico personalizado	`Case(When(prioridad='urgente', then=0), ...)`
Crear, editar y eliminar	`form.save()` y `objeto.delete()` desde ModelForms

3.8 Panel de control

El panel resume la operación con indicadores y gráficos que se filtran por período (7 días, 30 días, 90 días o 12 meses) y se comparan con el período anterior. Su contenido se guió por la imagen de referencia del caso, pero con un diseño propio: barra segmentada por estado, barras horizontales por tipo de vehículo, columnas de evolución, ranking de conductores y línea de tiempo de viajes. Cada gráfico tiene tooltip y "Ver como tabla".





Figura 9. Panel de control (período de 30 días).

4 Énfasis de la evaluación: autenticación y seguridad

Lo que se debe demostrar	Cómo se demuestra en el sistema	Prueba automática
1. Iniciar sesión correctamente	Login en /cuentas/login/ → panel con mensaje de bienvenida	`test_login_correcto_crea_sesion...`



Lo que se debe demostrar	Cómo se demuestra en el sistema	Prueba automática
2. Cerrar sesión	Botón "Cerrar sesión" (POST + CSRF) → login con mensaje	`test_logout_cierra_sesion...`
3. Vistas protegidas con decoradores	`@login_required` en cada vista; `@administrador_requerido` en usuarios	`test_operador_recibe_403`
4. Sin sesión no se accede por URL	Abrir /vehiculos/ sin sesión redirige al login con aviso	`test_vistas_protegidas_redirigen...` (recorre todas las URLs)
5. Tras el logout se pierde el acceso	Después de cerrar sesión, el panel vuelve a pedir login; "Atrás" no muestra datos	`test_logout_cierra_sesion_y_quita_el_acceso`
6. La sesión identifica al usuario	"Mi perfil" muestra el usuario, inicio de sesión, acceso anterior y expiración; envíos e incidencias guardan su autor	`test_registro_guarda_al_usuario...`

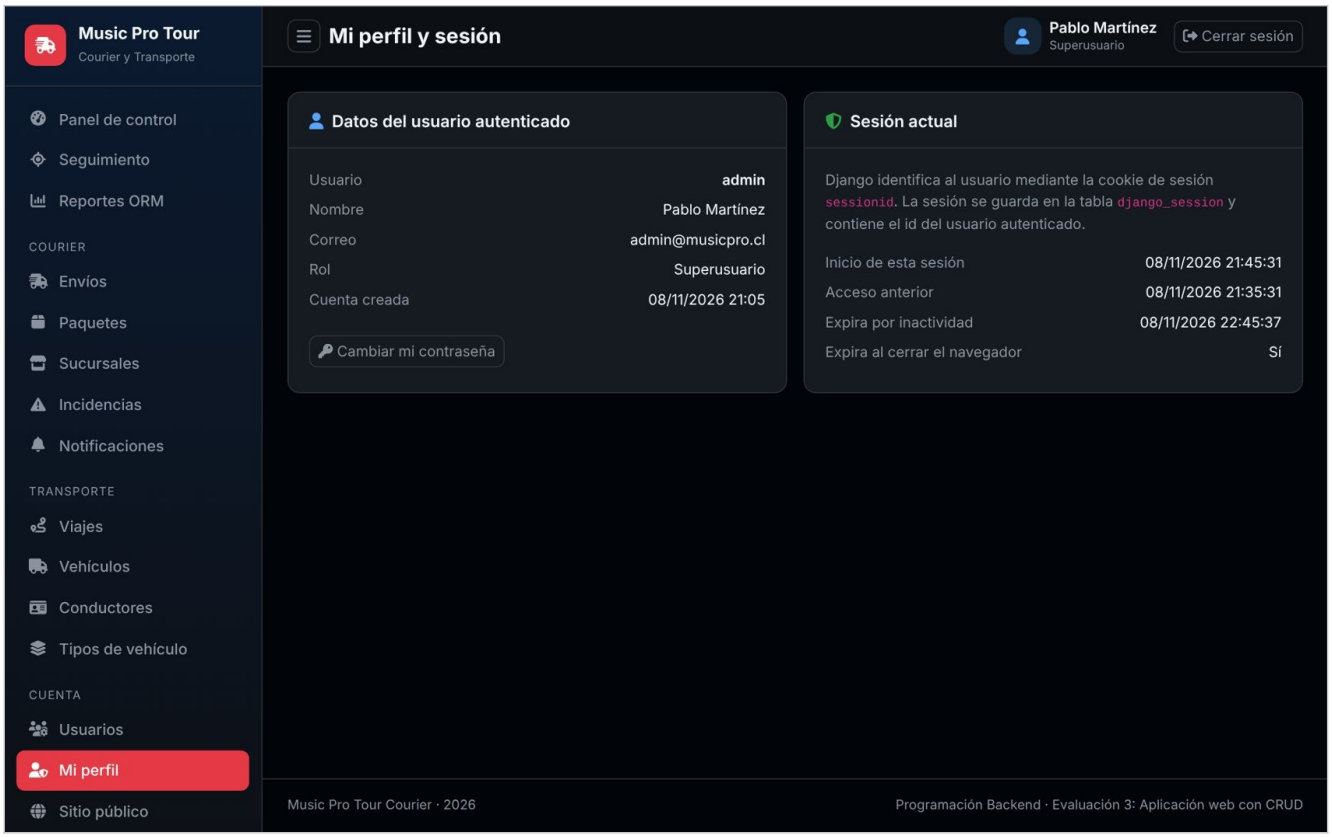


Figura 10. "Mi perfil": datos del usuario autenticado leídos desde la sesión.

5 Parte 2: Consultas con Django ORM

La página **Reportes ORM** (`/reportes/`) contiene los **20 reportes** del proyecto Transporte, agrupados por nivel. Así, cualquier selección de 10 consultas que entregue el docente ya está implementada. Cada reporte permite elegir parámetros (tipo, vehículo, conductor, rango de fechas o año) y muestra:

- el **resultado** en una tabla y, cuando corresponde, un gráfico;
- el **código ORM** exacto que se ejecutó (leído desde `courier/reportes.py`);



- el **SQL que generó Django** a partir de ese código, como evidencia de su funcionamiento.

N.º	Nivel	Pregunta	Técnica ORM
1	Básico	¿Cuáles son todos los vehículos registrados?	`select_related`, `order_by`
2	Básico	¿Qué vehículos pertenecen a un tipo seleccionado?	`filter(tipo_vehiculo=...)`
3	Básico	¿Qué vehículos están disponibles?	`filter(disponible=True)`
4	Básico	¿Cuáles son los 10 vehículos más antiguos?	`order_by('anio')[:10]`
5	Básico	¿Cuántos conductores están registrados?	`aggregate(Count)`
6	Intermedio	Viajes de un vehículo específico	`filter(vehiculo=...)`
7	Intermedio	Viajes asignados a un conductor	`filter(conductor=...)`
8	Intermedio	Viajes programados, en curso, finalizados o cancelados	`values_list().annotate(Count)`
9	Intermedio	Viajes en un rango de fechas	`fecha_salida__gte / __lt`
10	Intermedio	Vehículos sin viajes registrados	`filter(viajes__isnull=True)`
11	Intermedio	Cinco conductores con más viajes	`annotate(Count('viajes'))[:5]`
12	Intermedio	Cinco vehículos con más viajes	`annotate(Count('viajes'))[:5]`
13	Avanzado	Viajes por conductor en el último mes	`Count(..., filter=Q(...))`
14	Avanzado	Tipos de vehículo más utilizados	`Count('vehiculos__viajes')`
15	Avanzado	Viajes hacia cada destino	`values('destino').annotate()`
16	Avanzado	Conductores sin viajes en un período	`exclude(id__in=subconsulta)`
17	Avanzado	Duración promedio de los viajes finalizados	`Avg(F(...) - F(...))`
18	Análisis	Conductor con más viajes finalizados en el año	`Count(filter=Q(...__year=))`
19	Análisis	% de viajes finalizados, cancelados o pendientes	`aggregate` + porcentajes
20	Análisis	Distribución mensual de viajes en un año	Agregación condicional (12 `Count`)

Los rangos de fechas se calculan en Python con la zona horaria de Chile y se filtran con `__gte` / `__lt`. De esta forma los reportes funcionan aunque MySQL no tenga cargadas sus tablas de zonas horarias, lo habitual en Windows.



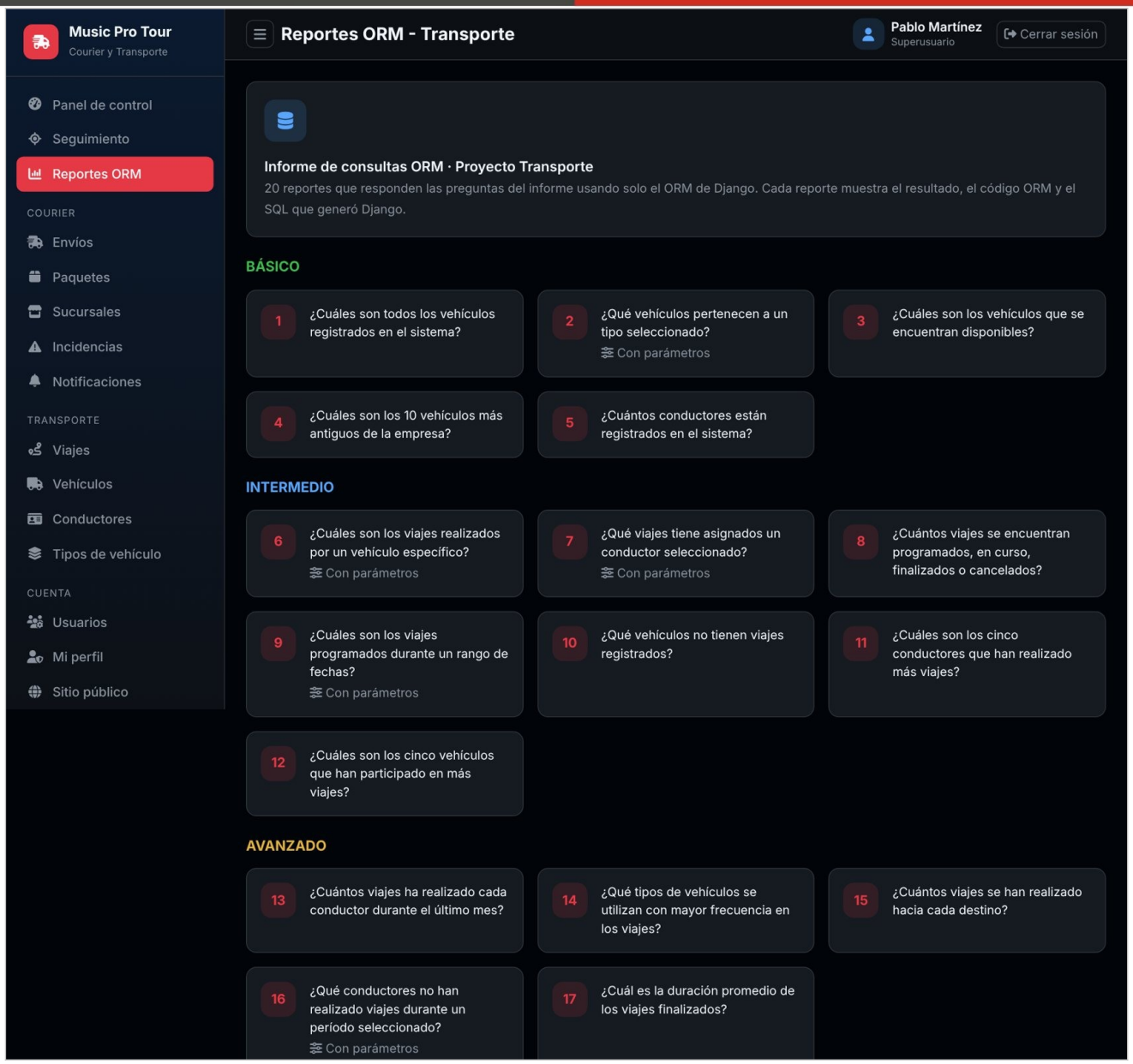


Figura 11. Índice de reportes agrupados por nivel.



Music Pro Tour
Courier y Transporte

Panel de control
Seguimiento
Reportes ORM

COURIER
Envíos
Paquetes
Sucursales
Incidencias
Notificaciones

TRANSPORTE
Viajes
Vehículos
Conductores
Tipos de vehículo

CUENTA
Usuarios
Mi perfil
Sitio público

Reporte 14

Pablo Martínez
Superusuario
Cerrar sesión

← Todos los reportes
Reporte 13 >
Reporte 15 >

14 AVANZADO
¿Qué tipos de vehículos se utilizan con mayor frecuencia en los viajes?

Resultado 5 filas

TIPO DE VEHÍCULO	VEHÍCULOS	VIAJES
Furgón	6	218
Camioneta	5	159
Camión	5	154
Minibús	3	99
Automóvil	3	25

Gráfico



▶ Ver como tabla

Consulta con Django ORM (courier/reportes.py)

```
@reporte(14, 'Avanzado', '¿Qué tipos de vehículos se u
def tipos_mas_utilizados(p):
    tipos = (TipoVehiculo.objects
             .annotate(total_viajes=Count('vehiculos__
                       vehiculos_distintos=Count('vehi
             .order_by('-total_viajes', 'nombre'))
    filas = [[t.nombre, t.vehiculos_distintos, t.total
    return Resultado(['Tipo de vehículo', 'Vehículos',
                      grafico={'tipo': 'ranking', 'etic
```

SQL generado por el ORM (1 consulta)

```
SELECT `courier_tipovehiculo`.`id`,
`courier_tipovehiculo`.`nombre`,
`courier_tipovehiculo`.`descripcion`,
COUNT(`courier_viaje`.`id`) AS `total_viajes`,
COUNT(DISTINCT `courier_vehiculo`.`id`) AS
`vehiculos_distintos` FROM `courier_tipovehiculo`
LEFT OUTER JOIN `courier_vehiculo` ON
(`courier_tipovehiculo`.`id` =
`courier_vehiculo`.`tipo_vehiculo_id`) LEFT OUTER
```

Music Pro Tour Courier · 2026

Programación Backend · Evaluación 3: Aplicación web con CRUD

Figura 12. Reporte 14: resultado, gráfico, código ORM y SQL generado por Django.



6 Modelo de datos

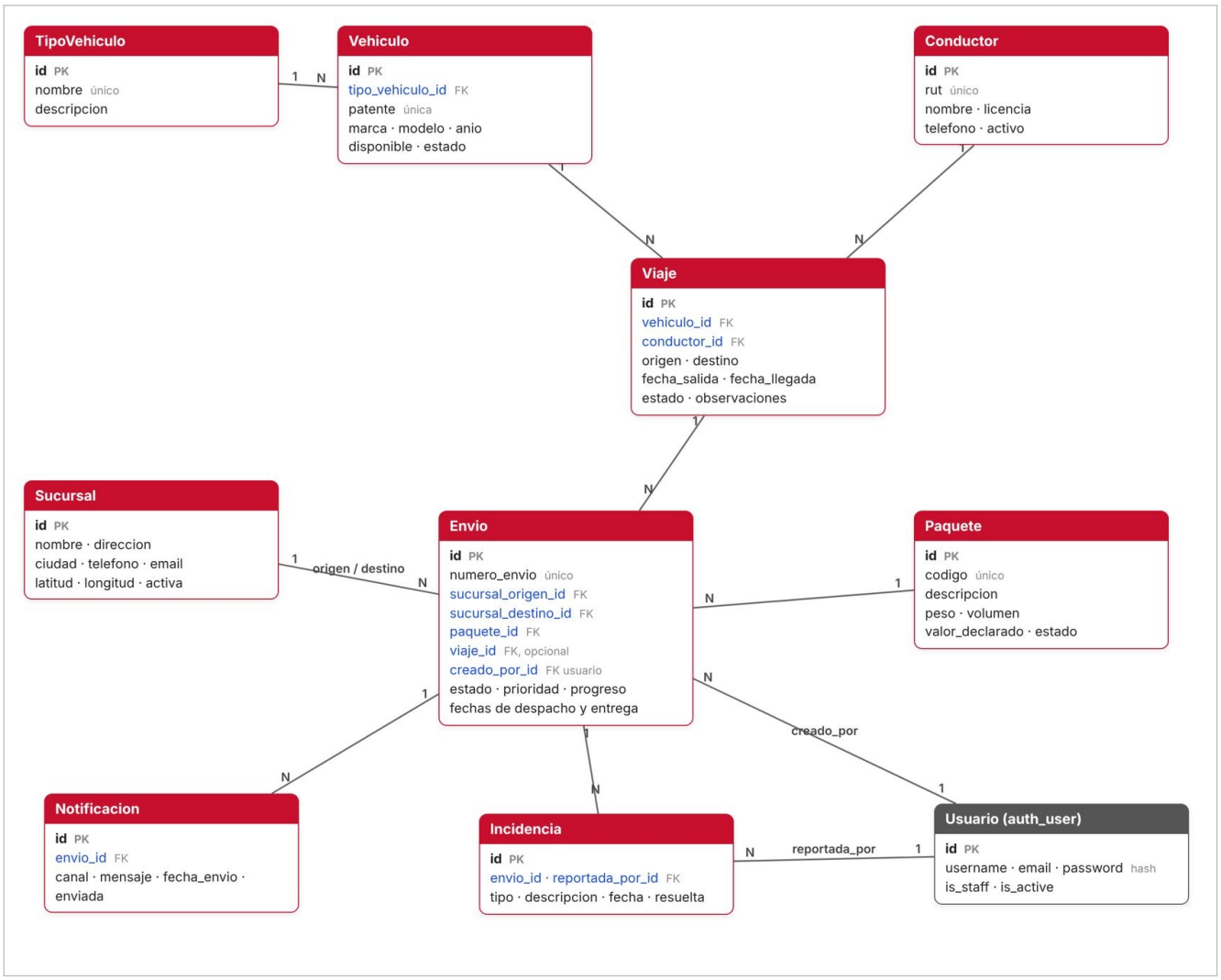


Figura 13. Modelo de datos: módulo Courier (Evaluación 2) y módulo Transporte (Evaluación 3).

- **TipoVehiculo 1–N Vehiculo**, **Vehiculo 1–N Viaje** y **Conductor 1–N Viaje**: base del proyecto Transporte.
- **Viaje 1–N Envio**: cada envío puede asignarse a un viaje (vehículo + conductor).
- **Sucursal 1–N Envio** (origen y destino) y **Paquete 1–N Envio**.
- **Envio 1–N Incidencia** y **Envio 1–N Notificacion**.
- **Usuario 1–N Envio / Incidencia**: el sistema registra qué usuario autenticado creó cada envío o reportó cada incidencia.
- Las claves foráneas principales usan `on_delete=PROTECT` para no perder información al eliminar.

7 Seguridad aplicada

- Contraseñas cifradas (PBKDF2) y validadores de Django para contraseñas débiles.



- Protección **CSRF** en todos los formularios; logout y desactivación de usuarios solo por POST.
- Renovación del id de sesión al iniciar sesión (evita la fijación de sesión).
- Sesión con expiración por inactividad (1 hora) y al cerrar el navegador; cookies HttpOnly y SameSite.
- Páginas privadas con `Cache-Control: no-store`: tras cerrar sesión no se ven con "Atrás".
- Control de acceso por rol (operador / administrador / superusuario) y redirección `next` segura.
- Clave secreta y credenciales en `.env`, fuera del código. Con `DEBUG=False` se exige `DJANGO_SECRET_KEY`.
- El ORM parametriza las consultas (previene inyección SQL) y las plantillas escapan el HTML.

8 Pruebas automáticas

El proyecto incluye **60 pruebas automáticas** (`python manage.py test courier`) que se ejecutaron con resultado exitoso tanto en SQLite como en MySQL (MariaDB). Verifican, entre otros puntos:

- login correcto e incorrecto, usuario desactivado, `next` seguro y cambio de id de sesión;
- que **todas** las URLs privadas redirigen al login sin sesión y que un POST sin sesión no modifica datos;
- que el logout solo acepta POST y quita el acceso;
- la mantención de usuarios y sus restricciones (operador recibe 403, nadie se desactiva a sí mismo, etc.);
- crear, editar y eliminar cada una de las 9 entidades, con su reflejo en la base de datos;
- las validaciones (RUT, patente, fechas, origen distinto del destino, registros protegidos);
- los 20 reportes y la exactitud de sus resultados, y el panel en cada período.

9 Uso de inteligencia artificial como apoyo

Se utilizó un asistente de IA (Claude) para analizar el enunciado, proponer la estructura, generar y revisar código y sugerir medidas de seguridad. Las recomendaciones se evaluaron antes de aplicarlas:

Recomendación	Decisión
Proteger las vistas con `@login_required` y un decorador propio para administradores	Aplicada
Cerrar sesión solo por POST con CSRF	Aplicada (además, Django 5+ lo exige)
Cabeceras `no-store` para que "Atrás" no muestre páginas tras el logout	Aplicada (middleware)
Credenciales en variables de entorno / `.env`	Aplicada
Usar `PROTECT` en las claves foráneas en vez de `CASCADE`	Aplicada: evita borrar envíos al eliminar una sucursal
Usar `__date` / `TruncMonth` para agrupar por fecha	Descartada: en MySQL sin tablas de zona horaria devuelven NULL; se usaron rangos y agregación condicional



Recomendación	Decisión
Bloquear cuentas tras varios intentos fallidos (django-axes)	No aplicada: agrega una dependencia fuera del alcance; queda como mejora futura
Vistas genéricas basadas en clases para todo el CRUD	Descartada: se prefirieron funciones con decoradores, que dejan explícito el control de acceso

10 Conclusión

Se desarrolló una aplicación web backend con Django y MySQL que reemplaza Django Admin por un administrador propio. El sistema autentica a los usuarios, protege cada funcionalidad con decoradores, identifica al usuario mediante la sesión y permite mantener tanto los usuarios como las entidades del caso Music Pro, con todas las operaciones realizadas a través del ORM. Además, responde las 20 preguntas del proyecto Transporte con consultas ORM cuya evidencia (resultado, código y SQL) se presenta en la página de reportes. Las 60 pruebas automáticas respaldan el funcionamiento de la autenticación, el control de acceso, el CRUD y los reportes.

Repositorio: **https://github.com/maxsteel19/musicpro-courier**



APRENDER HACIENDO A LO LARGO DE LA VIDA

ACREDITADO

7 AÑOS
MÁXIMA
EXCELENCIA

CNA
Comisión Nacional
de Acreditación
Chile

INSTITUTO PROFESIONAL INACAP
ACREDITADO • NIVEL DE EXCELENCIA
• Gestión Institucional • Docencia de Pregrado
• Vinculación con el Medio
• Próxima Acreditación Septiembre 2030

www.inacap.cl